Задача 1: Пример за сериализация с интерфейс Serializable

Условие: Създайте клас Person, който съдържа име и възраст на човек, и имплементира интерфейса Serializable. След това сериализирайте обект на Person в файл.

Обяснение:

- Класът Person имплементира интерфейса Serializable, което означава, че може да бъде сериализиран.
- Обектът на клас Person се записва в файл чрез ObjectOutputStream.

```
import java.io.*;  // Импортиране на библиотеката за работа с входно-изходни операции

    // Клас Person, който имплементира интерфейса Serializable, за да може обектите му
да бъдат сериализирани
    class Person implements Serializable {
        // Полета за име и възраст на лицето
        private String name;
        private int age;

        // Конструктор за инициализиране на полетата name и age
        public Person(String name, int age) {
            this.name = name;
            this.age = age;
        }

        // Гетър за получаване на името
        public String getName() {
            return name;
        }

        // Гетър за получаване на възрастта
        public int getAge() {
            return age;
        }

        // Метод toString за представяне на обекта като текст
        @Override
        public String toString() {
            return "Person{name='" + name + "', age=" + age + "}";
        }
    }

    // Главен клас за демонстриране на сериализацията
    public class SerializationDemo {
        public static void main(String[] args) {
            // Създаваме обект от тип Person с име "John" и възраст 25
            Person person = new Person("John", 25);

            // Използваме блок try-with-resources за автоматично затваряне на ресурсите
            try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("person.ser"))) {
                // Сериализираме обекта person и го записваме във файл person.ser
                oos.writeObject(person);
                // Показваме съобщение за успешно сериализиране
                System.out.println("Object serialized successfully.");
            } catch (IOException e) {
                // Ако възникне грешка по време на сериализацията, отпечатваме
информация за грешката
                e.printStackTrace();
            }
        }
    }
```

Задача 2: Пример за сериализация с интерфейс Externalizable

Условие: Създайте клас Employee, който имплементира Externalizable и запазва информация за служителите.

```
import java.io.*;  // Импортиране на библиотеката за работа с входно-изходни операции

// Клас Employee, който имплементира интерфейса Externalizable
class Employee implements Externalizable {
    // Полета за име и заплата на служителя
    private String name;
    private double salary;

    // Празен конструктор, който е необходим при използване на Externalizable
    public Employee() {
        // Този конструктор е задължителен, за да може обектът да бъде възстановен по
време на десериализация
    }

    // Конструктор за инициализиране на полетата name и salary
    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    // Метод за записване на обекта във файл (сериализация)
    @Override
    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeObject(name);  // Записваме името като обект
        out.writeDouble(salary);  // Записваме заплатата като double
    }

    // Метод за прочитане на обекта от файл (десериализация)
    @Override
    public void readExternal(ObjectInput in) throws IOException, ClassNotFoundException
{
        name = (String) in.readObject();  // Четем името като обект и го преобразуваме
в String
        salary = in.readDouble();  // Четем заплатата като double
    }

    // Метод за текстово представяне на обекта Employee
    @Override
    public String toString() {
        return "Employee{name='" + name + "', salary=" + salary + "}";
    }
}

// Главен клас за демонстриране на сериализацията и десериализацията
public class ExternalizableDemo {
    public static void main(String[] args) {
        // Създаваме обект от тип Employee с име "Alice" и заплата 50000
        Employee employee = new Employee("Alice", 50000);

        // Сериализация на обекта Employee
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("employee.ser"))) {
            oos.writeObject(employee);  // Записваме обекта в employee.ser
            System.out.println("Employee object serialized successfully.");
        } catch (IOException e) {
            e.printStackTrace();  // Отпечатваме стека на грешката при IOException
        }

        // Десериализация на обекта Employee
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("employee.ser"))) {
            Employee deserializedEmployee = (Employee) ois.readObject();  // Четем
```

```
обекта от файла
            System.out.println("Deserialized Employee: " + deserializedEmployee);  //
Извеждаме обекта
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();  // Отпечатваме стека на грешката при IOException или
ClassNotFoundException
        }
    }
}
```

Задача 3: Примерен код за обработка на грешки

```
// Импортираме необходимите библиотеки от пакета java.io за работа с файлове и
управление на изключенията
import java.io.*;

public class FileErrorHandlingExample {
    public static void main(String[] args) {
        // Блокът try се използва за опит да изпълним код, който може да предизвика
изключение
        try {
            // Опитваме се да създадем FileReader за четене на файл с име
"nonexistentfile.txt"
            FileReader reader = new FileReader("nonexistentfile.txt");

            // Четем един символ от файла. Ако файлът не съществува, се хвърля
FileNotFoundException
            int data = reader.read();
        }
        // Обработваме изключението, което се хвърля, ако файлът не е намерен
        catch (FileNotFoundException e) {
            // Извеждаме съобщение, че файлът не е намерен
            System.out.println("The file was not found.");
        }
        // Обработваме общо изключение IOException, което може да се появи при проблеми
с четенето на файла
        catch (IOException e) {
            // Извеждаме съобщение за грешка при четене на файла
            System.out.println("An error occurred while reading the file.");
        }
    }
}
```

Задача 4: Примерен код за създаване на файл

```
import java.io.File;
import java.io.IOException;

public class FileCreationExample {
    public static void main(String[] args) {
        // Създаваме обект от тип File, който сочи към нов файл с име "newfile.txt"
        File file = new File("newfile.txt");

        // Блокът try се използва, за да се опитаме да създадем файла и да обработим
потенциални грешки
        try {
            // Проверяваме дали файлът вече съществува
            if (file.exists()) {
                System.out.println("File already exists."); // Извеждаме съобщение, ако
файлът вече съществува
            } else {
                // Опитваме се да създадем нов файл
                if (file.createNewFile()) {
                    System.out.println("File created successfully."); // Извеждаме
съобщение, ако файлът е създаден успешно
                } else {
                    System.out.println("File creation failed."); // Извеждаме
съобщение, ако създаването на файла не е успешно
                }
            }
        } catch (IOException e) {
            // Обработваме изключението, което може да възникне при създаването на
файла
            System.out.println("An error occurred while creating the file.");
            e.printStackTrace(); // Извеждаме стека на грешката за по-подробна
информация
        }
    }
}
```

Задача 5: Копиране на съдържание на файл с FileInputStream и FileOutputStream

```
import java.io.*;

public class CopyFile {
    public static void main(String args[]) throws IOException {
        FileInputStream in = null; // Декларираме променлива от тип FileInputStream за
четене от файл
        FileOutputStream out = null; // Декларираме променлива от тип FileOutputStream
за записване във файл

        try {
            // Отваряме файла "input.txt" за четене
            in = new FileInputStream("input.txt");
            // Създаваме или отваряме файла "output.txt" за записване
            out = new FileOutputStream("output.txt");

            int c; // Декларираме променлива за съхраняване на байта, който четем
            // Четем от "input.txt" и записваме всеки байт в "output.txt"
            while ((c = in.read()) != -1) {
                out.write(c); // Записваме байта във файла "output.txt"
            }
        } finally {
            // Затваряме файловете, за да освободим ресурсите
            if (in != null) {
                in.close(); // Затваряме FileInputStream
            }
            if (out != null) {
                out.close(); // Затваряме FileOutputStream
            }  } }}
```

Задача 6: Копиране съдържание на файл с FileReader и FileWriter

```
import java.io.*;

public class CopyFile {
    public static void main(String args[]) throws IOException {
        FileReader in = null; // Декларираме променлива от тип FileReader за четене на текстови файлове
        FileWriter out = null; // Декларираме променлива от тип FileWriter за записване в текстови файлове

        try {
            // Отваряме файла "input.txt" за четене
            in = new FileReader("input.txt");
            // Създаваме или отваряме файла "output.txt" за записване
            out = new FileWriter("output.txt");

            int c; // Декларираме променлива за съхраняване на символа, който четем
            // Четем от "input.txt" и записваме всеки символ в "output.txt"
            while ((c = in.read()) != -1) {
                out.write(c); // Записваме символа във файла "output.txt"
            }
        } finally {
            // Затваряме файловете, за да освободим ресурсите
            if (in != null) {
                in.close(); // Затваряме FileReader
            }
            if (out != null) {
                out.close(); // Затваряме FileWriter
            }
        }
    }
}
```

Задача 7: Запис на масив от байтове в текстов файл.

```
mport java.io.*;

public class CopyFile {

    public static void main(String args[]) {

        try {
            // Създаваме масив от байтове, който ще записваме в файла
            byte[] writeArr = {1, 2, 3, 4, 5};

            // Създаваме обект за изходен поток, който ще записва в "test.txt"
            OutputStream os = new FileOutputStream("test.txt");

            // Използваме цикъл, за да запишем всички байтове от масива в файла
            for (int i = 0; i < writeArr.length; i++) {
                os.write(writeArr[i]);  // Записваме всеки байт поотделно в потока
            }

            // Затваряме изходния поток след като всички байтове са записани
            os.close();

            // Създаваме обект за входен поток, който ще чете от "test.txt"
            InputStream is = new FileInputStream("test.txt");

            // Получаваме размера на наличните байтове в потока
            int size = is.available();

            // Четем всички байтове от входния поток и ги отпечатваме като символи
            for (int i = 0; i < size; i++) {
                // Четем по един байт и го преобразуваме в символ, който да отпечатаме
```

```
                System.out.print((char) is.read() + "  ");
            }

            // Затваряме входния поток след като сме прочели всички данни
            is.close();

        } catch (IOException e) {
            // В случай на грешка при работа с файловете, извеждаме съобщение за
изключение
            System.out.print("Exception");
        }
    }
}
```

Задача 8: Пример за BufferedReader и BufferedWriter

```
import java.io.*;

public class BufferedStreamExample {
    public static void main(String[] args) {
        try {
            // Буфериран входен поток за четене на текстови данни
            BufferedReader reader = new BufferedReader(new FileReader("input.txt"));
            // Буфериран изходен поток за писане на текстови данни
            BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"));

            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line);
                writer.newLine(); // добавя нов ред след всяка линия
            }

            reader.close();
            writer.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```